

Abstract

- The agenda of this project is to analyze authentication mechanisms used in web applications, focusing on session management.
- The study examines a web application that uses **JWT tokens** instead of traditional session IDs and highlights how improper handling of authentication tokens can lead to risks similar to session hijacking.
- A detailed Proof of Concept (PoC) conducted in a **controlled lab environment using a deliberately vulnerable web application** which demonstrates how JWT exposure can result in unauthorized access.
- The project concludes with an impact analysis and recommended mitigation strategies to secure session and token-based authentication mechanisms.

Name: OWASP Juice Shop

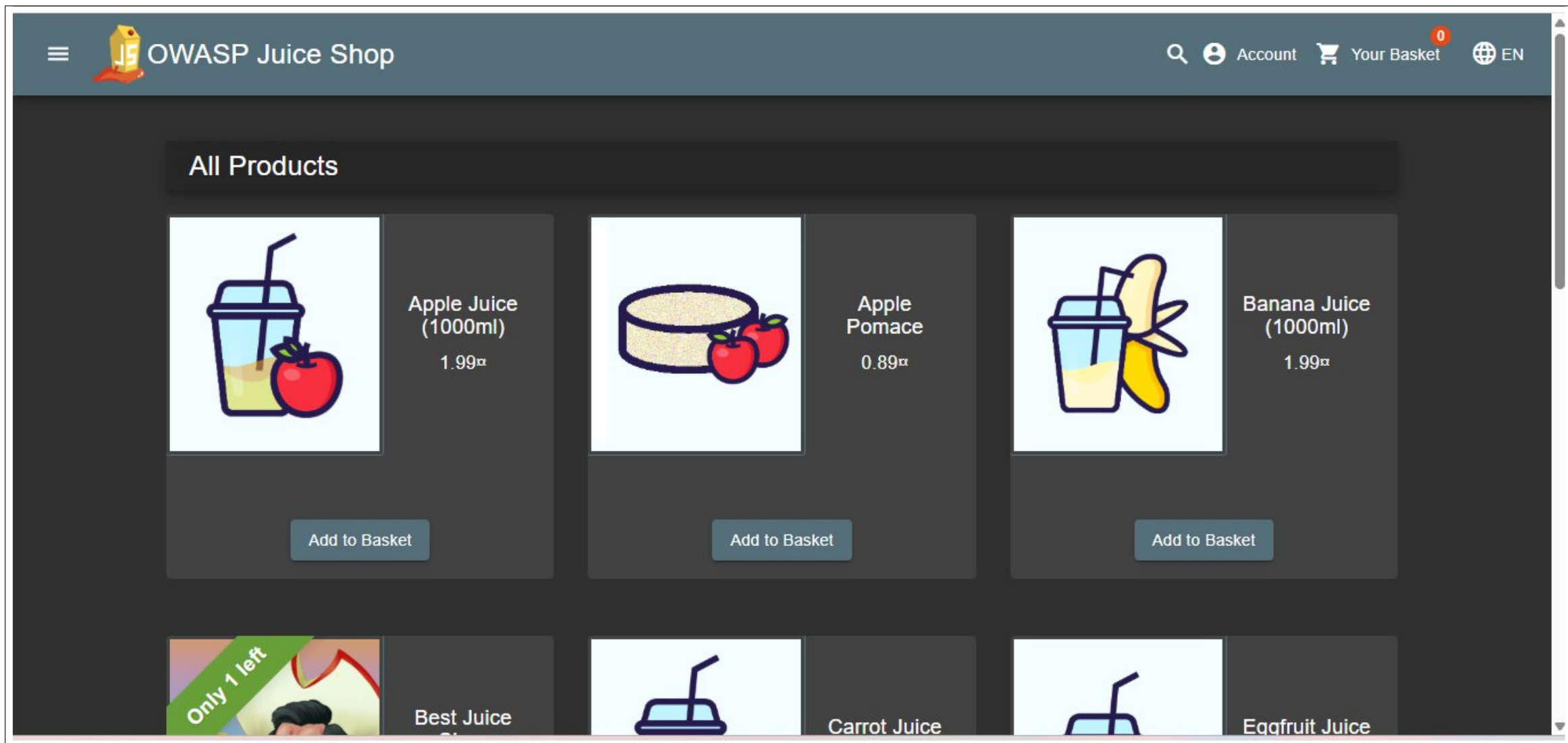
URL: <https://owasp.org/www-project-juice-shop/> (http://localhost:3000)

Category/Type: Vulnerable E-Commerce Web Application (Refer picture [1])

Usage/Popularity:

- OWASP flagship project used globally for security training
- Widely used by students & professionals covers OWASP Top 10 vulnerabilities actively maintained

Vulnerable Web Application Details



Picture [1]

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

Component	Technology Used	How to Identify
Frontend	Angular, JS, HTML, CSS	Page source, DevTools
Backend	Node.js (Express.js)	HTTP Headers
Database	SQLite	Official documentation
API Type	REST APIs	Network traffic analysis
Authentication	JWT Tokens	Local Storage, Authorization header

Vulnerable Website

- Data Collection

- Tools used:**
- Docker
 - Browser DevTools
 - BurpSuite
 - JWT.io

Cookies and Sessions

- Cookies are small pieces of data stored in the user's browser. They help to remember things like login status or preferences even after closing the website.
- Cookies usually contain the session ID that links the user to their server-side session.
- Whenever the user sends a request, the browser automatically sends the cookie to the server, allowing the server to verify the user's identity.
- Sessions store user data on the server, making them more secure and ideal for storing temporary or sensitive information.
- The server creates a session for each logged-in user and assigns a unique session ID. This session ID acts as a temporary identity card that allows the server to recognize the user across multiple requests. Without sessions, users would have to log in again for every page they visit.

Cookies and Sessions

- Sessions and cookies work together to maintain user authentication.
- The session is stored on the server, while the session ID is stored in the user's browser as a cookie.
- The cookie does not store the user's password but only a random identifier.
- If an attacker steals this session ID cookie, they can impersonate the user without knowing their login credentials.

Why Session IDs and Cookies Are Sensitive

- Session IDs are equivalent to temporary passwords. anyone who possesses a valid cookie can access the user's account.
- If cookies are not properly protected, attackers can steal them using techniques such as Cross-Site Scripting (XSS), malware, or network sniffing. This can result in session hijacking, account takeover, and data theft.

Insecure Token – JWT Based Authentication

- In the selected web application session IDs are replaced with JWT tokens. While traditional session hijacking via cookies may not apply, JWT tokens introduce equivalent risks when stored or handled insecurely.
- Since OWASP Juice Shop uses **JWT-based authentication** instead of traditional server-side sessions, classic session hijacking via cookie reuse is not applicable. Instead, the risk lies in JWT token exposure, where a stolen token can be reused until its expiration.
- This vulnerability directly impacts the confidentiality and integrity of user accounts and aligns with **OWASP Top 10** issues related to **broken authentication and session management**.

Common Session, Token & Cookie Vulnerabilities

- Insecure cookie flags - Missing **HttpOnly**, **Secure**, **SameSite** flag
- Session Hijacking
- Session Fixation
- XSS-based cookie theft

These weaknesses make session cookies vulnerable to theft and misuse.

If Cookies and tokens Are Compromised

- Unauthorized account access
- Data leakage
- Privilege escalation
- Complete user impersonation

Business Impact

- Loss of user trust
- Compliance violations
- Financial loss
- Legal consequences

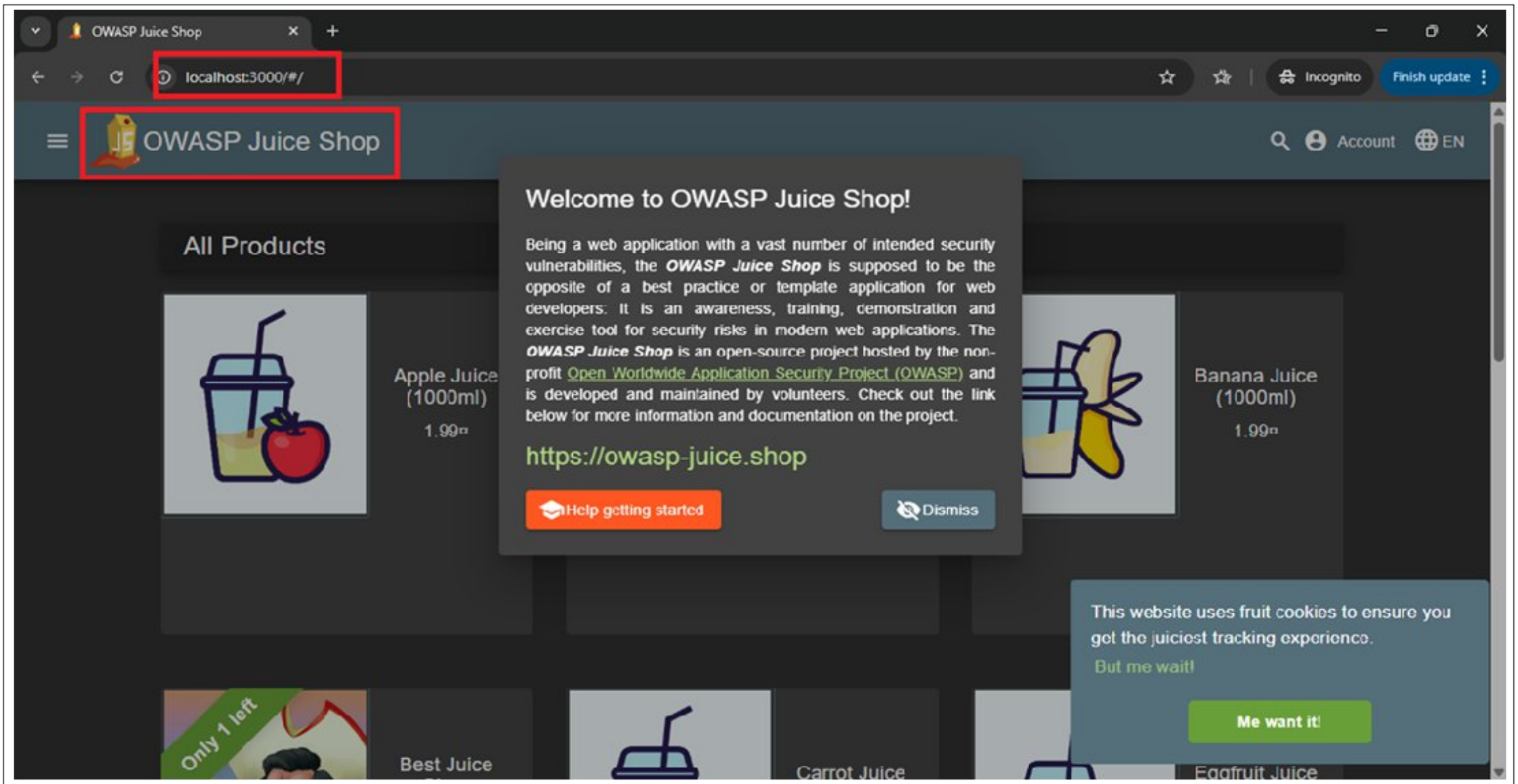
Proof of Concept (PoC)

Identified Vulnerability - **Reflected Cross-Site Scripting (XSS)**

Step 01: Run OWASP Juice Shop

- Make sure Docker is running
- OWASP Juice Shop was deployed locally to ensure a controlled and testing environment.
- Open Command Prompt and run below command
 - i. `docker pull bkimminich/juice-shop`
 - ii. `docker run -d -p 3000:3000 bkimminich/juice-shop`

Open browser and visit: <http://localhost:3000> (Refer picture[2])

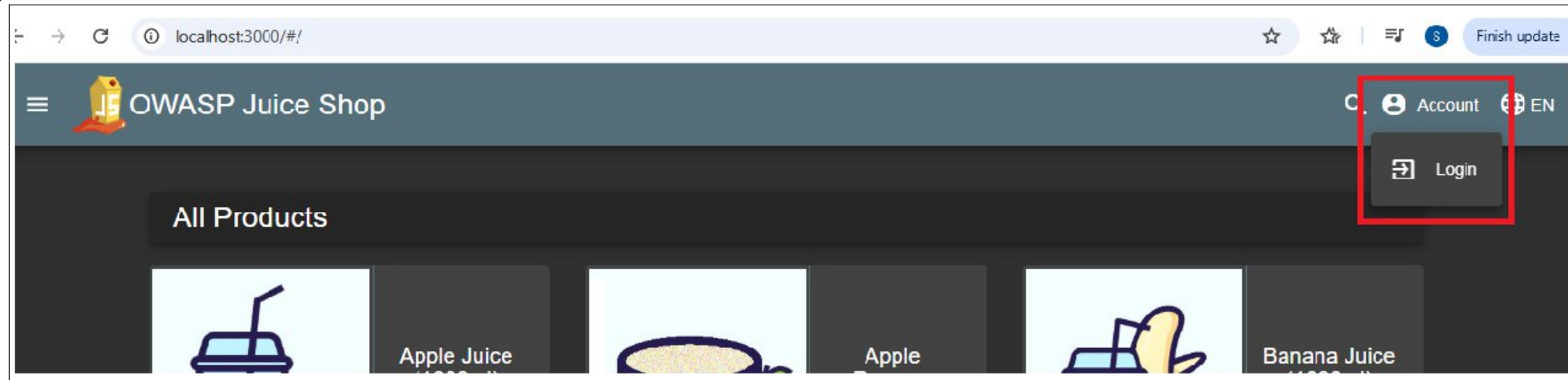


Picture [2]

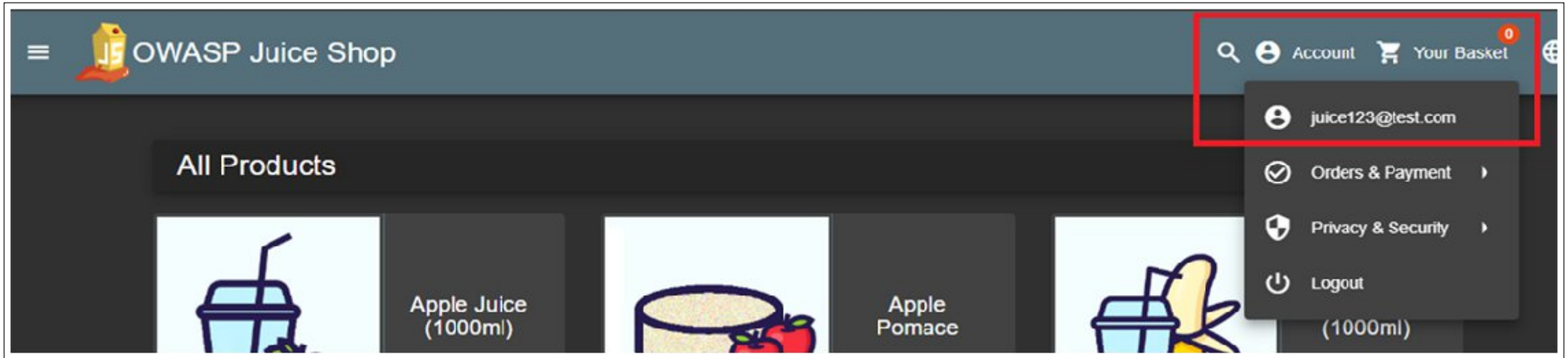
Step 02: Create Test User

- Click Account → Login
- Click “Not yet a customer?”
- Register:
 - i. **Email:** juice123@test.com
 - ii. **Password:** project
- Login (Refer picture[3 & 4])

Picture [3]



Picture [4]



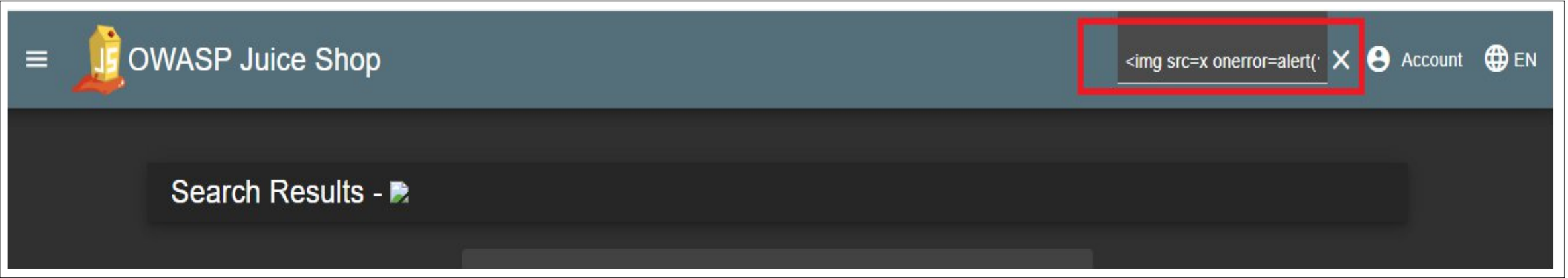
Step 03: Check for reflected XSS vulnerability

To confirm Reflected XSS Vulnerability, type below command on “Search” panel

- `<img src=x onerror=alert(1)>`

Expected output: A popup appears with 1 (Refer Picture[5 & 6])

Picture [5]



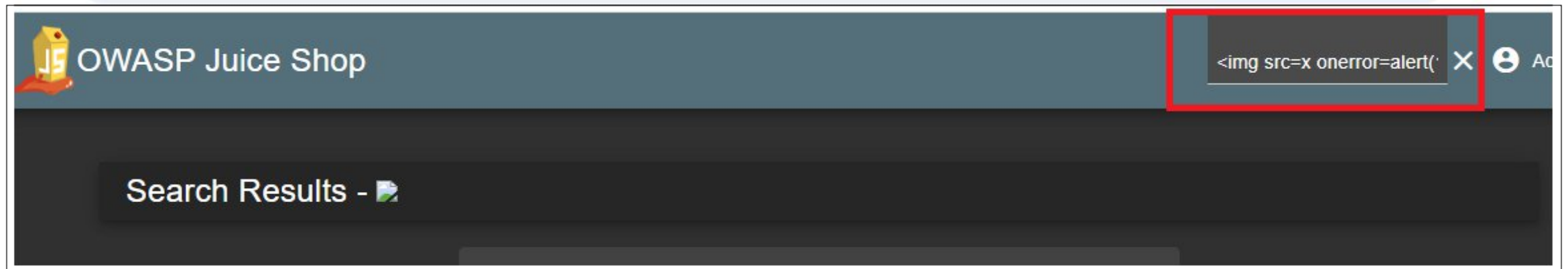
Picture [6]



As we got the expected output try below command to get JWT token of the login user

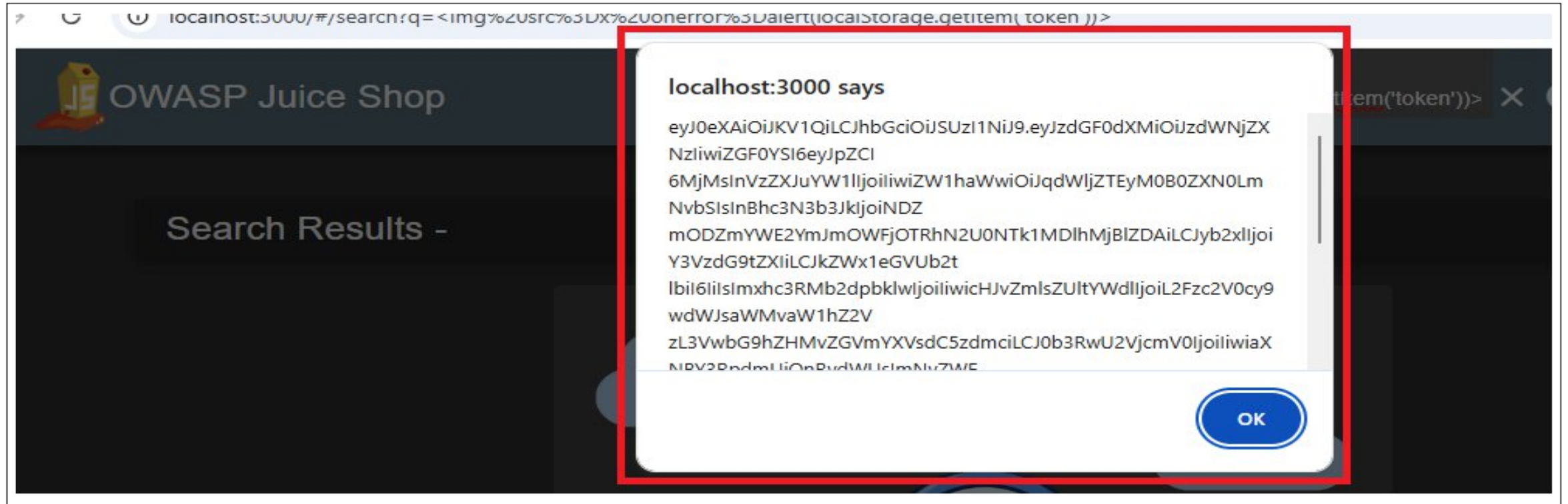
- `<img src=x onerror=alert(localStorage.getItem('token'))>`

Expected output: Alert popup with **long JWT token** (Refer picture [7 & 8])



Picture [7]

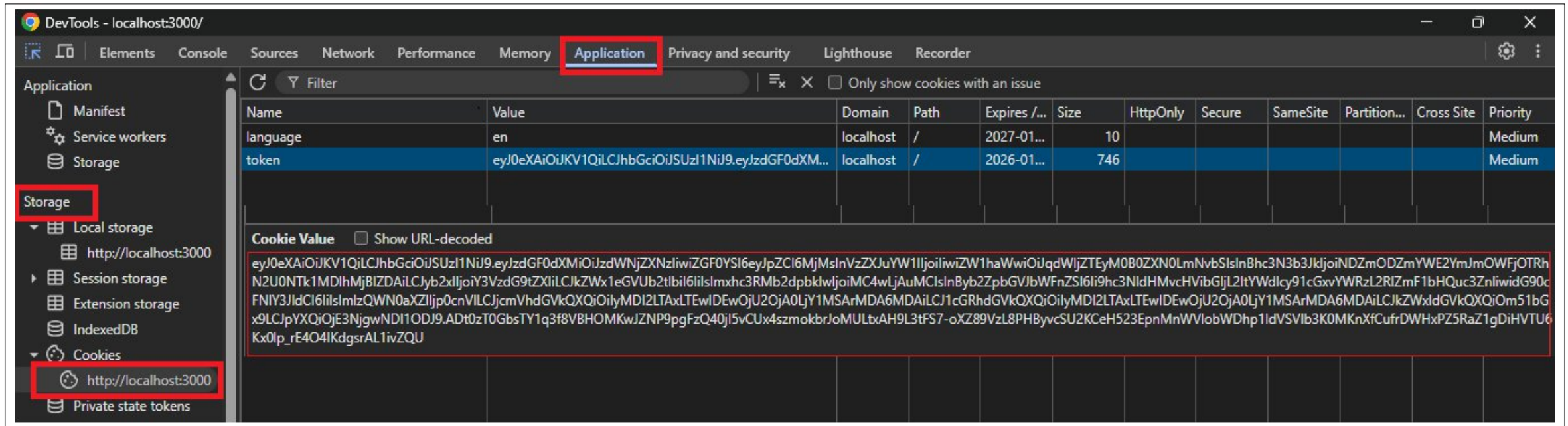
Picture [8]



Can also be validated in BrowserDev tools (F12)

- Press **F12**
- Go to **Application → Local Storage**
- Show **token** value (Refer picture [9])

Picture [9]



Step 04: Exploiting JWT token using Burp Suite

- Open OWASP juice shop and login over Burp Suite browser
- Observe HTTP traffic and find login request to get JWT authentication token
- PFB screenshot for reference (Refer picture [10])

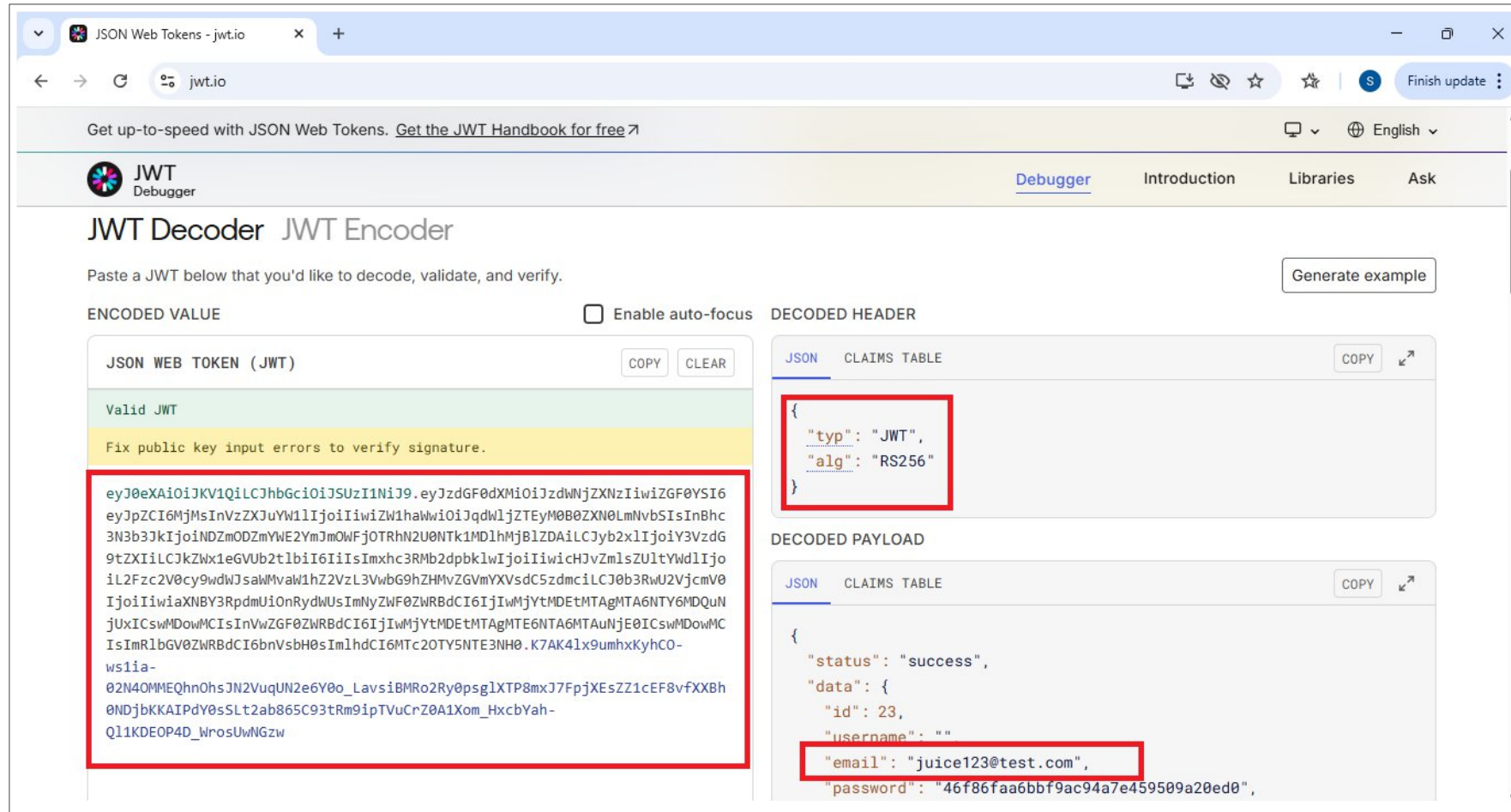
Picture [10]

[illegible]

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

- Sent the HTTP request to repeater and copy the authentication token.
- Open jwt.io to decode the JWT token. (Refer picture [11 &12])

Picture [11]



- Here we can manipulate the email and algorithm and encode the JWT authentication token.
- Change the email to ["jwt3d@juice-sh.op."](#) and algorithm to **"none"**.
- And copy the generated JWT token (Refer picture [12])

JWT Decoder

JWT Encoder

Fill in the fields below to generate a signed JWT.

HEADER: ALGORITHM & TOKEN TYPE

CLEAR

Valid header

```
{  "typ": "JWT",  "alg": "none"}  
```

PAYLOAD: DATA

CLEAR

Valid payload

```
{  "status": "success",  "data": {    "id": 23,    "username": "",    "email": "jwtn3d@juice-sh.op",    "password": "40f80fdd8bbf3ac34d7c459509a20ed0",    "role": "customer"  }  }
```

JSON WEB TOKEN

COPY

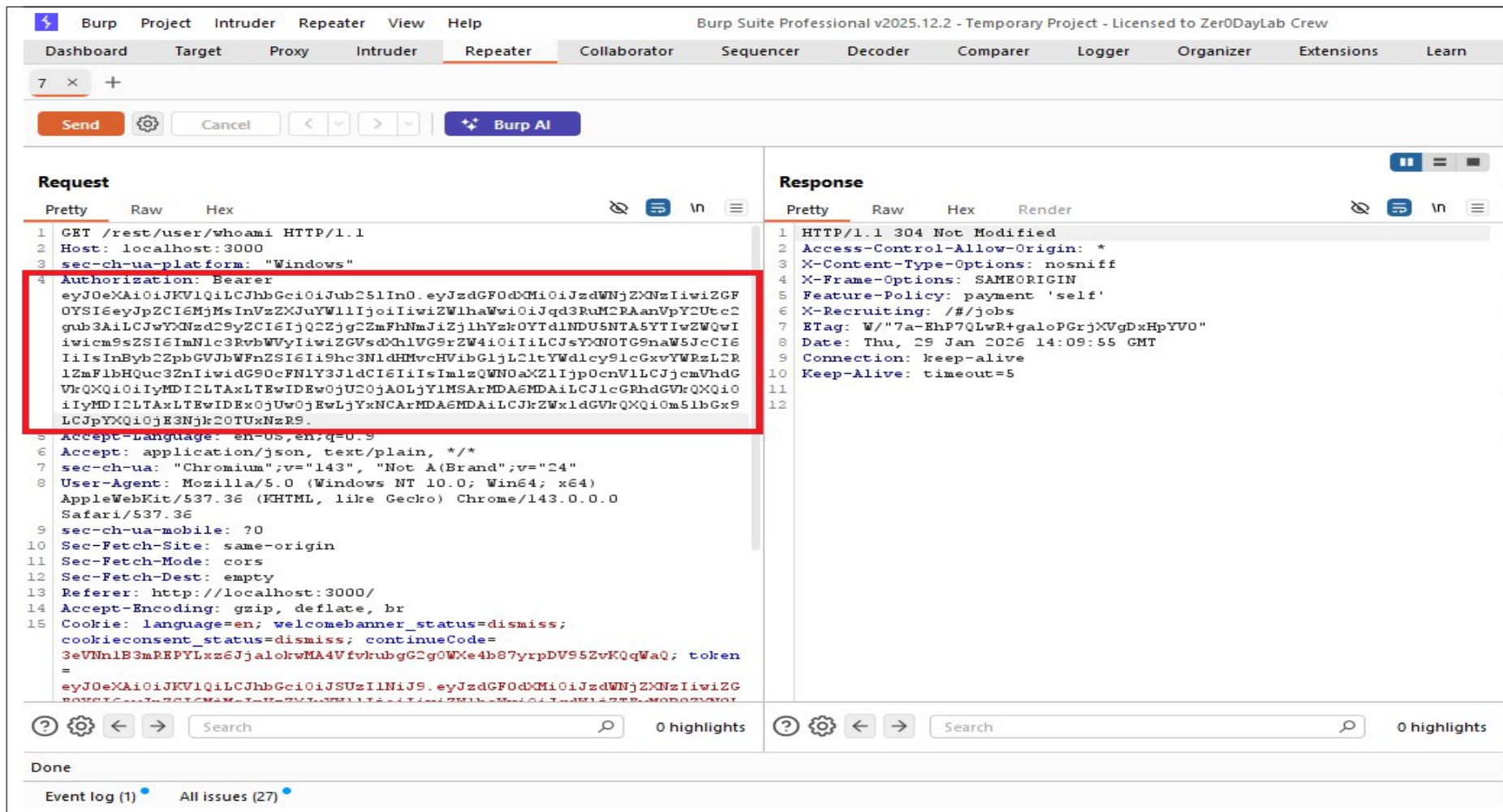
This is an Unsecured JWT as defined by [Section 6 of RFC 7519](#).

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIub251In0.eyJzdGF0dXMiOiJzdWVjZXNzIiwiaWZGF0YSI6eyJ3pZCI6MjMsInVzZXJuYVllIjoiiIiwiaWZlbnR1bGwiOiJqd3RUM2RAanVpY2Utc2gub3AiLCJwYXNzd29yZCI6IjQ2Zjg2ZmFhNmJiZjJlYzkyYTdlNDU5NTA5YTIwZWQwIiwicm9sZSI6ImN1c3RvbWVyIiwiaWZGVsdXhlVG9rZW4iOiIiLCJzYXN0TG9naW5JcCI6IiIsInByb2ZpbGVJbWFnFnZSI6Ii9hc3NldHMvcHVibG1jL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNLy3JldCI6IiIsIm1zQWNoZXZlIjpb0cnVlLCJjcmVhdGVkQXQxOiIiYMDI2LTaxLTewIDEwOjU2OjA0LjY1MSArMDA6MDAiLCJ1cGRhdGVkQXQxOiIiYMDI2LTaxLTewIDEwOjUwOjEwLjYxNCArMDA6MDAiLCJkZWxlZGVkQXQxOiI0M51bGx9LCJpYXQxOiI0jE3Njk20TUxNzR9.
```

Picture [12]

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

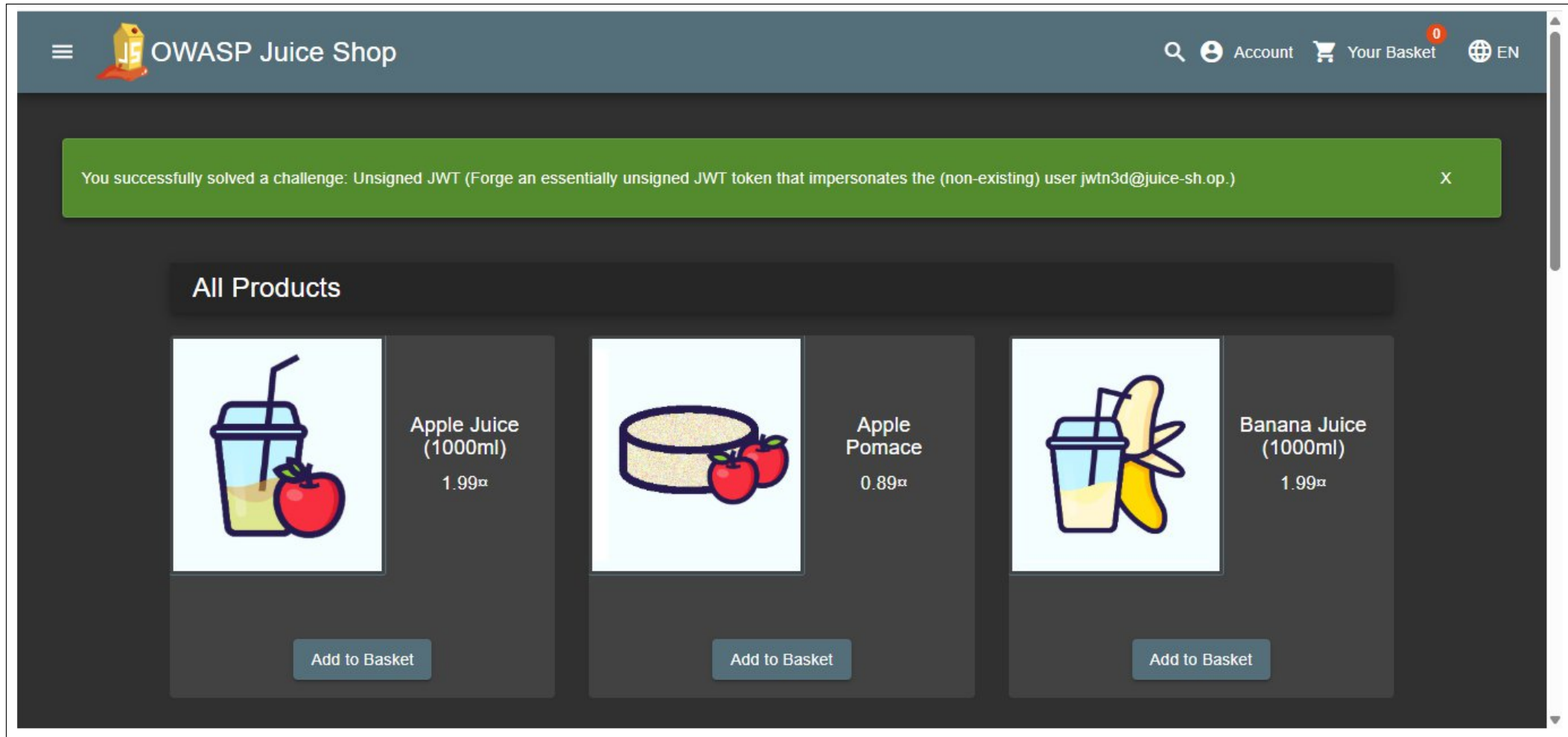
- Paste it in authentication header of Burp suite request and refresh the browser. (Refer picture [13])



Picture [13]

- An unsigned JWT token that impersonates the (non-existing) user jwtn3d@juice-sh.op. (Refer picture [14])

Picture [14]



- This vulnerability allows an attacker to forge and reuse unsigned JSON Web Tokens, resulting in complete authentication bypass. By hijacking and manipulating JWT claims, an attacker can impersonate any user, including privileged accounts, without knowing valid credentials. This represents a critical JWT hijacking flaw caused by improper token verification.

Recommendations & Mitigations

- Avoid storing JWT in Local Storage and Only explicitly approved algorithms (e.g., RS256 / HS256) should be allowed.
- Use **HttpOnly Secure, SameSite** flags
- Implement token expiration & rotation
- Regenerate session IDs after login
- Explicitly blacklist the **none** algorithm at the application and framework level.

Conclusion

- The security risks associated with session ID and cookie-based authentication mechanisms using the OWASP Juice Shop web application as a case study.
- Due to improper validation of JWT signatures (**alg: none**), the application blindly trusts attacker-supplied tokens. This enables JWT hijacking through token forgery, allowing unauthorized account takeover and privilege escalation without password compromise.
- This issue falls under **OWASP Top 10 – Broken Authentication**, where improper JWT signature verification allows token hijacking and user impersonation. An attacker can fully bypass authentication controls by crafting unsigned JWTs.
- The project highlights that **secure implementation is critical for any authentication mechanism**, and effective protections such as **XSS prevention, secure token storage, and defence-in-depth controls** are essential to reduce overall security risk.

CONFIDENTIAL: The information in this document belongs to Boston Institute of Analytics LLC. Any unauthorized sharing of this material is prohibited and subject to legal action under breach of IP and confidentiality clauses.

References

- <https://owasp.org/www-project-juice-shop/>
- <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/XSS%20Injection>
- <https://jwt.io/introduction>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>
- https://owasp.org/www-community/attacks/Session_hijacking_attack